

1η Προγραμματιστική Εργασία (Σε ομάδες το πολύ 2 ατόμων)

Βάρος: 25% βαθμού μαθήματος

Παράδοση: έως 03/04/2026, ώρα 23:55

Οδηγίες: Ηλεκτρονική παράδοση συμπιεσμένου αρχείου μέσω του ιστοχώρου του μαθήματος (συμπεριλαμβανομένου και του κώδικα). Φροντίστε να είναι καλά οργανωμένο το παραδοτέο σας σε υποκαταλόγους με ευνόητα ονόματα. Τεκμηριώστε τον κώδικα σας επαρκώς. Συνοδέψτε την εργασία με μία το πολύ 12-σέλιδη αναφορά όπου εξηγείτε τις επιλογές σας και περιγράφετε τα αποτελέσματα σας. Θα βαθμολογηθείτε και για την ποιότητα της αναφοράς σας - φροντίστε λοιπόν να είναι κατανοητή, να απαντάει τα ερωτήματα της εκφώνησης, και να αντιστοιχεί σε / περιγράφει ορθά τον κώδικά σας.

Μπορείτε ελεύθερα να χρησιμοποιείτε έτοιμο κώδικα για επιμέρους κομμάτια των προγραμμάτων σας αρκεί να αναφέρετε επακριβώς τις πηγές σας.

Κατά τα λοιπά, αντιγραφή συνεπάγεται άμεσο μηδενισμό στο μάθημα.

Σε αυτή την εργασία καλείστε να λύσετε το πρόβλημα εύρεσης βέλτιστης διαδρομής που αντιμετωπίζει ένα όχημα, το οποίο πρέπει να μεταβεί από μία αρχική θέση σε ένα τελικό στόχο αποφεύγοντας κάποια εμπόδια. Το πρόγραμμα σας θα πρέπει να λύνει το πρόβλημα με τρεις διαφορετικούς τρόπους χρησιμοποιώντας τους παρακάτω αλγορίθμους αναζήτησης:

(Α) τον αλγόριθμο απληροφόρητης αναζήτησης *Iterative-Deepening DFS*

(Β) τον αλγόριθμο πληροφορημένης αναζήτησης *Weighted A**

(Γ) τον αλγόριθμο online αναζήτησης *LRTA**

Στις περιπτώσεις (Α) και (Β), το όχημα λύνει το πρόβλημα *offline* πριν την αναχώρησή του, και ακολουθεί το πλάνο διαδρομής που του προτείνει το πρόγραμμά σας. Στην περίπτωση (Γ), το όχημα χρησιμοποιεί την τρέχουσα κατάσταση της διαδρομής και λύνει το πρόβλημα online - δηλαδή δε γνωρίζει το κόστος κάθε μονοπατιού παρά μόνο αφού το διασχίσει, και λαμβάνει αποφάσεις για τοπικές κινήσεις.

Το πρόγραμμά σας θα δέχεται ως είσοδο ένα αρχείο σχετικό με ένα σενάριο, το οποίο περιλαμβάνει πληροφορίες για την αρχική θέση του οχήματος, τα όρια του δρόμου, τα εμπόδια και τον τελικό στόχο του οχήματος. Σκοπός σας είναι να καθοδηγήσετε το όχημα ως τον τελικό προορισμό του, αποφεύγοντας να χτυπήσει κάποιο εμπόδιο ή να βρεθεί εκτός των ορίων του δρόμου.

Σας δίνεται μια πλατφόρμα υλοποιημένη σε Python 3.9, που αυτοματοποιεί αρκετές λειτουργίες, όπως το visualization κάθε σεναρίου και των βημάτων που εκτελεί ο εκάστοτε αλγόριθμος, αλλά

και της τελικής διαδρομής του οχήματος. Η πλατφόρμα αυτή είναι διαθέσιμη στον αντίστοιχο φάκελο “Εγγραφα” του eclass. Επιπλέον, σας δίνεται ένα παράδειγμα υλοποίησης του αλγορίθμου DFS (προσοχή, όχι του Iterative-Deepening DFS) και templates κώδικα για τον κάθε αλγόριθμο που καλείστε να υλοποιήσετε.

Για την περίπτωση **(B)**, η συνάρτηση εκτίμησης που θα χρησιμοποιήσετε για τον weighted A* ορίζεται ως $f(n) = g(n) + w * h(n)$. Η συνάρτηση κόστους $g(n)$ δίνεται έτοιμη σε συνάρτηση μέσα στα αρχεία που καλείστε να συμπληρώσετε! Αυτή η συνάρτηση δίνει το κόστος του μονοπατιού σε μονάδα μέτρησης απόστασης. Το βάρος w θα πρέπει να παραμένει το ίδιο καθ’ όλη τη διάρκεια εκτέλεσης κάθε προσομοίωσης και καλείστε να πειραματιστείτε με διαφορετικές τιμές ($w_1 = 0, w_2 = 1 \dots$). Τέλος, θα πρέπει να υλοποιήσετε 2 ευρετικές συναρτήσεις $h(n)$: η πρώτη θα στηρίζεται στην Ευκλείδεια απόσταση, ενώ τη δεύτερη θα την επιλέξετε εσείς και θα πρέπει να εξηγήσετε στην αναφορά σας τους λόγους επιλογής της καθώς και το πως την κατασκευάσατε.

Για τις εκδοχές της περίπτωσης **(B)** με $w > 1$, θα πρέπει να πειραματιστείτε σε όλα τα σενάρια με διάφορες τιμές του w και να καταλήξετε σε μία τιμή w ανά σενάριο.

Το πρόγραμμά σας θα πρέπει να εμφανίζει στην οθόνη αλλά και να παράγει ένα αρχείο εξόδου (βλέπε Παράρτημα Γ) όπου θα αναφέρονται για κάθε αλγόριθμο και για κάθε διαφορετικό σενάριο που σας δίνεται:

- ο αριθμός κόμβων που επεκτείνονται (Visited nodes number),
- το ακριβές δρομολόγιο (ακολουθία κόμβων) που προτείνει ο αλγόριθμος να ακολουθήσει το όχημα (Path) και
- το εκτιμώμενο από τον αλγόριθμο συνολικό κόστος $g(n)$ της τελικής διαδρομής.

Παραδοτέα Εργασίας (σε ένα αρχείο μορφής .zip):

1. Τελικός κώδικας της υλοποίησης σας
2. Αναφορά

Μαζί με το κώδικα σας θα πρέπει να παραδώσετε και μία αναφορά στην οποία: **(i)** να εξηγήσετε τον τρόπο επιλογής και τις ιδιότητες του τελικού w για κάθε σενάριο στην περίπτωση (B) και να σχολιάσετε την απόδοση του αλγορίθμου για τις διαφορετικές τιμές που δοκιμάσατε (καλείστε να παραθέσετε κατάλληλους πίνακες/γραφήματα που να παρουσιάζουν τα αποτελέσματα των πειραμάτων σας για τις διάφορες τιμές του w). **(ii)** να περιγράψετε τα πειράματα που τρέξατε για τον κάθε αλγόριθμο, να παρουσιάσετε σε πίνακα τον συνολικό αριθμό βημάτων για κάθε παραλλαγή, και να εξάγετε συμπεράσματα από αυτά: Ποιος αλγόριθμος δίνει καλύτερα αποτελέσματα; Σε ποιους τομείς; Ποια η εξήγησή σας για την παρατηρούμενη συμπεριφορά; Πως συμπεριφέρεται ο LRTA* σε σχέση με τους offline αλγορίθμους; Μην ξεχάσετε να συγκρίνετε όλους τους αλγορίθμους μεταξύ τους (και μη ξεχάσετε να τεκμηριώσετε την επιλογή

της δεύτερης ευρετικής συνάρτησης - βλέπε παραπάνω). Μετά από κάθε επιτυχή εκτέλεση ενός αλγορίθμου, το πρόγραμμα αποθηκεύει ένα τελικό στιγμιότυπο στον φάκελο *Figures*, εκ των οποίων να συμπεριλάβετε μόνο αξιοσημείωτα παραδείγματα στην αναφορά σας. Προσοχή: Ο κώδικας σας θα πρέπει να είναι εκτελέσιμος και θα πρέπει να περιλάβετε οδηγίες εγκατάστασης και εκτέλεσης στην αναφορά σας. Σε περίπτωση που χρησιμοποιήσετε επιπλέον βιβλιοθήκες θα πρέπει να ανανεώσετε το αρχείο requirements.txt που βρίσκετε στα αρχεία που σας δίνονται για την εργασία, ώστε να εγκαθίστανται αυτόματα οι βιβλιοθήκες αυτές. Αυτό, μπορεί να γίνει μέσω της εντολής `pip freeze`.

Οδηγίες Παράδοσης:

Στην περίπτωση που η ομάδα αποτελείται από δυο (2) άτομα, μόνο το **ένα** από αυτά χρειάζεται να ανεβάσει στο eclass τα παραδοτέα. Προσοχή: στην αναφορά πρέπει να αναγράφονται τα ονόματα όλων των μελών της ομάδας

Βαθμολογία (25% βαθμού μαθήματος):

- Έως 15% του βαθμού για την υλοποίηση των τριών αλγορίθμων (5% ανά αλγόριθμο)
- Έως 10% του βαθμού για την αναφορά
- Σε περίπτωση που ο κώδικας σας δε μπορεί να εκτελεστεί θα υπάρχει απώλεια 17/25 μονάδων από το βαθμό της εργασίας σας.

Παράρτημα Α: Οδηγίες Εγκατάστασης

Ο κώδικας υλοποίησης των αλγορίθμων θα πρέπει να είναι σε Python 3.9, σύμφωνα με τα template που σας δίνονται. Συστήνεται η χρήση της [Python 3.9.13](#) και η χρήση virtual environment (πχ, μέσω του `venv` ή της πλατφόρμας του [Anaconda/Miniconda](#)).

Πιο συγκεκριμένα, εφόσον εγκαταστήσετε την πλατφόρμα του Anaconda, μπορείτε να δημιουργήσετε ένα virtual environment για Python 3.9 , δίνοντας του ένα δικό σας όνομα (πχ “virtual_environment_name”), εκτελώντας την εξής εντολή:

```
conda create -n virtual_environment_name python=3.9.13 -c conda-forge -y
```

Στη συνέχεια (και κάθε φορά που ανοίγετε το terminal), θα πρέπει να “ενεργοποιείτε” το virtual environment που έχετε δημιουργήσει χρησιμοποιώντας την παρακάτω εντολή:

```
conda activate virtual_environment_name
```

Τέλος, θα πρέπει να εκτελέσετε και την παρακάτω εντολή ώστε να εγκαταστήσετε τις απαραίτητες βιβλιοθήκες που απαιτούνται για να λειτουργήσει σωστά ο κώδικας της εργασίας:

```
pip install -r requirements.txt
```

Προκειμένου να τρέξετε το πρόγραμμά σας εκτελέστε την εξής εντολή:

```
python main.py
```

Παράρτημα Β: Αρχεία προς τροποποίηση

Τα αρχεία που πρέπει να τροποποιήσετε περιλαμβάνουν το *main.py* (στο root φάκελο της εργασίας) και τα αρχεία στον φάκελο *Algorithms* (καθένα από αυτά αντιστοιχεί σε έναν αλγόριθμο που πρέπει να υλοποιήσετε). Στα αρχεία υπάρχουν **ήδη** τα templates των κλάσεων που πρέπει να υλοποιηθούν. Κάθε κλάση, κληρονομεί μεθόδους από την κλάση **Algorithms.Utils.SequentialSearch** οι οποίες μπορούν να χρησιμοποιηθούν και στις υλοποιήσεις σας.

Μέσω του αρχείου *main.py* μπορείτε να επιλέξετε το τρέχον σενάριο για τον κώδικα σας (δείτε τον φάκελο *Scenarios*) και αντίστοιχα να επιλέγετε ποιος/-οι αλγόριθμοι θα χρησιμοποιηθεί/-ούν. Εξετάστε προσεκτικά τις συναρτήσεις που σας δίνονται καθώς είναι υπεύθυνες για την ομαλή λειτουργία και σύνδεση με τις υπάρχουσες βιβλιοθήκες. Τέλος, στο φάκελο *Algorithms* (αρχείο *DFS_example.py*) σας δίνετε έτοιμη μια αναδρομική υλοποίηση του αλγορίθμου απληροφόρητης αναζήτησης *DFS*.

Παράρτημα Γ: Έξοδος προγράμματος (κονσόλα και αρχείο)

Η έξοδος του προγράμματος σας θα πρέπει να έχει την παρακάτω μορφή (αφορά και την έξοδο στην κονσόλα, αλλά και το αρχείο που θα πρέπει να παράγει το πρόγραμμα σας):

=====

<Scenario Name 1> // name of the first scenario

Iterative-Deepening DFS:

Visited Nodes number: x

Path: (x1,y1)->(x2,y2)->...->(xn,yn)

Estimated Cost: x

Weighted A* ($w=0$):

Visited Nodes number: x

Path: $(x_1, y_1) \rightarrow (x_2, y_2) \rightarrow \dots \rightarrow (x_n, y_n)$

Estimated Cost: x

Weighted A* ($w=1$):

Visited Nodes number: x

Path: $(x_1, y_1) \rightarrow (x_2, y_2) \rightarrow \dots \rightarrow (x_n, y_n)$

Estimated Cost: x

Weighted A* ($w=w_3$):

Visited Nodes number: x

Path: $(x_1, y_1) \rightarrow (x_2, y_2) \rightarrow \dots \rightarrow (x_n, y_n)$

Estimated Cost: x

.
.
.

LRTA*:

Visited Nodes number: x

Path: $(x_1, y_1) \rightarrow (x_2, y_2) \rightarrow \dots \rightarrow (x_n, y_n)$

Estimated Cost: x

<Scenario Name 2>

Iterative-Deepening DFS:

Visited Nodes number: x

Path: $(x_1, y_1) \rightarrow (x_2, y_2) \rightarrow \dots \rightarrow (x_n, y_n)$

Estimated Cost: x

Weighted A* ($w=0$):

Visited Nodes number: x

Path: $(x_1, y_1) \rightarrow (x_2, y_2) \rightarrow \dots \rightarrow (x_n, y_n)$

Estimated Cost: x

Weighted A* ($w=1$):

Visited Nodes number: x

Path: $(x_1, y_1) \rightarrow (x_2, y_2) \rightarrow \dots \rightarrow (x_n, y_n)$

Estimated Cost: x

Weighted A* ($w= w_3$):

Visited Nodes number: x

Path: $(x_1, y_1) \rightarrow (x_2, y_2) \rightarrow \dots \rightarrow (x_n, y_n)$

Estimated Cost: x

.

.

.

LRTA*:

Visited Nodes number: x

Path: $(x_1, y_1) \rightarrow (x_2, y_2) \rightarrow \dots \rightarrow (x_n, y_n)$

Estimated Cost: x

.

.

.

κ.ο.κ., για όλα τα σενάρια που σας παρέχονται.

Σας επισημαίνεται πως η εργασία πιθανότατα θα εξεταστεί και προφορικά σε ημερομηνία που θα προσδιοριστεί αργότερα.

Καλή δουλειά και καλή επιτυχία!